Spring Boot 小游戏
Docker 容器化实验报告

记忆花园 memory-game6085

课程：2026 云计算技术与架构

姓名：周宇斌

班级：软件 2401

学号：20246085

实验日期：2026 年 10 月 21 日



图 1　记忆花园容器运行页面

本实验将 Vue3、Spring Boot 与 SQLite 构成的记忆翻牌小游戏打包为 Docker 镜像，在本机私有 Registry 完成发布与重新拉取，并使用拉取的镜像创建新容器。新旧容器挂载同一命名数据卷，成绩记录保持一致。报告中的终端画面与游戏页面均来自实际运行。

目录

一 实验目的与任务要求 2
二 实验环境与项目结构 3
三 系统设计与持久化 4
3.1 应用结构 4
3.2 SQLite 数据设计 4
3.3 容器数据卷 4
四 本地构建与功能验证 5
4.1 本地构建与自动验证 5
4.2 容器中的真实游戏操作 5
五 Docker 镜像构建与运行 7
5.1 多阶段构建 7
5.2 启动容器与接口检查 9
六 镜像发布与拉取 10
6.1 启动本机镜像仓库 10
6.2 标记与推送镜像 10
6.3 删除本地镜像后重新拉取 11
七 容器重建与持久化验证 13
7.1 记录重建前的数据 13
7.2 使用拉取镜像创建新容器 14
7.3 对比重建后的成绩 14
八 实验结果与问题分析 16
8.1 启动与端口问题 16
8.2 实验总结 16
九 参考资料与证据来源 16

一 实验目的与任务要求

本实验的目的是掌握 Spring Boot 应用容器化的基本过程，理解 Docker 镜像、容器、镜像仓库和持久化数据卷的作用，并通过实际命令和访问结果验证部署流程。

原始作业要求来自 raw/要求.png：在机器上部署 Docker 环境，将一个已有 Spring Boot 项目打包到 Docker 上发布，项目名称末尾加学号后四位；发布后演示拉取，所有过程截图放在一个 Word 文件中提交。

任务要求	本实验对应方式
项目名称包含学号后四位	memory-game6085，学号后四位为 6085
Spring Boot 项目打包发布	三阶段 Dockerfile 构建，运行阶段提供页面和 API
发布后演示拉取	发布到 localhost:5000；删除本地镜像后重新 pull
全过程截图汇总	真实 Windows 终端截图、容器页面及持久化对比截图

二 实验环境与项目结构

项目	本次实际配置
操作系统	Windows 11 家庭版 中文版 64 位，10.0.26300
本地开发工具	Java 21.0.11；Node.js 24.14.1；npm 11.11.0；Maven 3.9.16
应用技术栈	Vue 3.5.44、Vite 7.3.7、Spring Boot 4.1.1、SQLite JDBC 3.53.4.0
Docker 环境	Docker Desktop 4.95.0；Client / Server 29.8.2；Compose 5.6.0
构建与运行镜像	Node 24；Maven 3.9.9 + Java 21；Temurin 21 JRE
源码目录	Code/memory-game6085/
应用端口	宿主机 127.0.0.1:18085 → 容器 8080
镜像仓库	本机 Registry，127.0.0.1:5000
持久化目录	memory-game6085-data → /app/data，数据库 garden.db

frontend 保存 Vue 页面和图案，backend 保存 Spring Boot 接口及数据库逻辑，scripts 保存本地构建与实验辅助脚本。Dockerfile 与 compose.yaml 位于项目根目录。8080 被已有课堂项目占用，因此本实验采用 18085 作为宿主机端口。

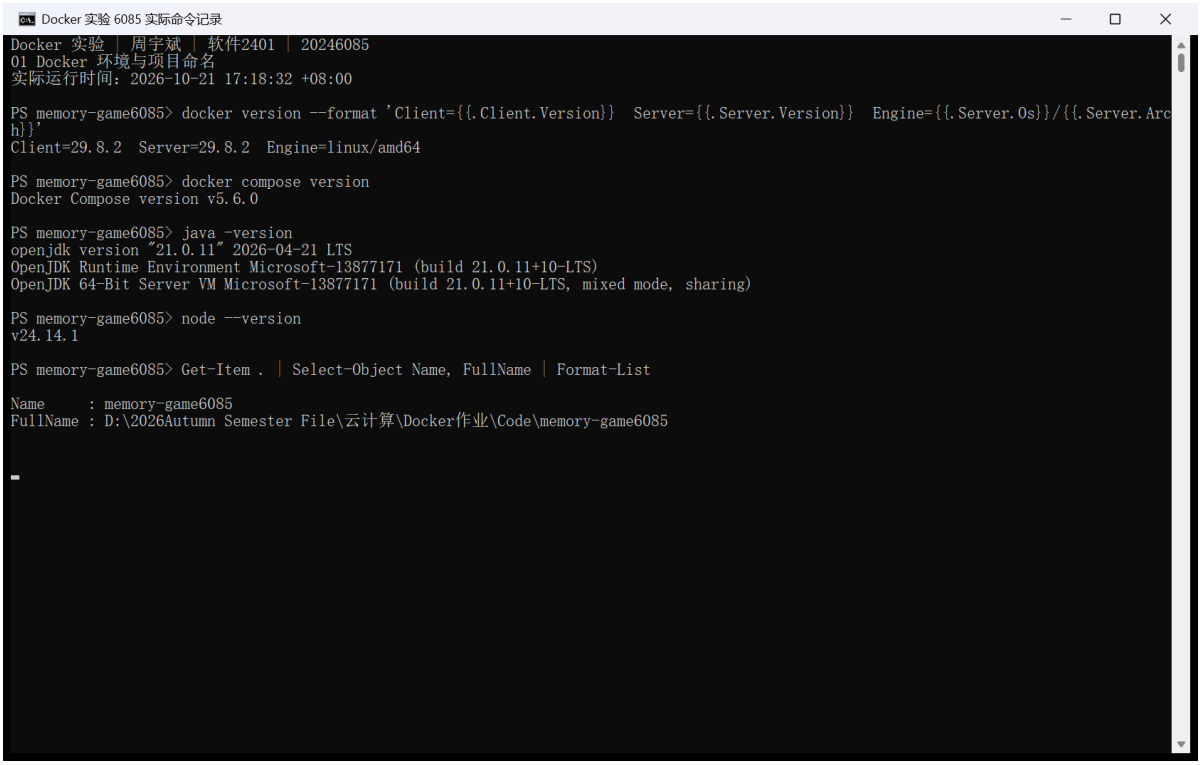


图 2　Docker Client 与 Server 版本及项目名称检查

三 系统设计与持久化

3.1 应用结构

浏览器通过 Vue3 页面操作游戏，Spring Boot 提供牌局、翻牌、暂停、成绩提交和排行榜接口。前端构建结果复制到后端静态资源目录，再打入可执行 JAR，因此一个应用容器即可同时提供页面和接口。

游戏提供轻松 6 对、标准 8 对和挑战 12 对三种难度。后端随机生成卡牌顺序并判断配对，负责步数和计时。未翻开的答案不会直接传给浏览器；暂停时保留本局状态并停止计时，恢复后继续游戏。

3.2 SQLite 数据设计

字段	用途
game_id	牌局唯一标识，防止同一局重复写入
nickname / difficulty	玩家昵称与难度，按难度分别查询排行榜
moves / duration_ms	服务端生成的步数与用时，作为排序依据
created_at	成绩记录创建时间

运行中的牌局保存在服务端内存中；服务重启后需要重新开局。已提交成绩保存在 SQLite 数据库，排行榜按步数升序排列，步数相同再比较用时。持久化对象是已保存成绩，不包括未完成牌局。

3.3 容器数据卷

```
volumes:
  - garden-data:/app/data
# 命名数据卷的实际名称
name: memory-game6085-data
```

镜像保存应用程序，数据库由运行时生成并写入 /app/data。该目录挂载到命名数据卷后，删除应用容器不会自动删除成绩文件。实验重建容器时继续挂载同一数据卷，以验证程序与数据的生命周期可以分离。

四 本地构建与功能验证

4.1 本地构建与自动验证

```
.\scripts\build.ps1
java -jar backend/target/memory-game6085.jar
```

本地构建已生成可执行 JAR。后端测试覆盖隐藏牌面、无效操作、暂停状态、服务端成绩统计和 SQLite 重新打开后的数据保留。浏览器测试覆盖完整配对、昵称校验、成绩提交、榜单刷新、窄屏布局以及接口失败后的重试。

检查项目	结果
npm run build	成功生成前端静态资源
后端测试	3 项通过，0 失败，0 错误
浏览器端到端测试	3 项全部通过
界面检查	1440、768、375、812 横屏及 320 宽度无横向溢出
数据保存	完成牌局后可提交昵称并进入对应难度榜单

4.2 容器中的真实游戏操作

在实际应用容器中通过浏览器完成轻松难度一局，保存昵称“6085 实验”。本次演示记录为 12 步、6.95 秒，记录编号 1。这些数值来自服务端实际返回，仅用于展示成绩保存流程。

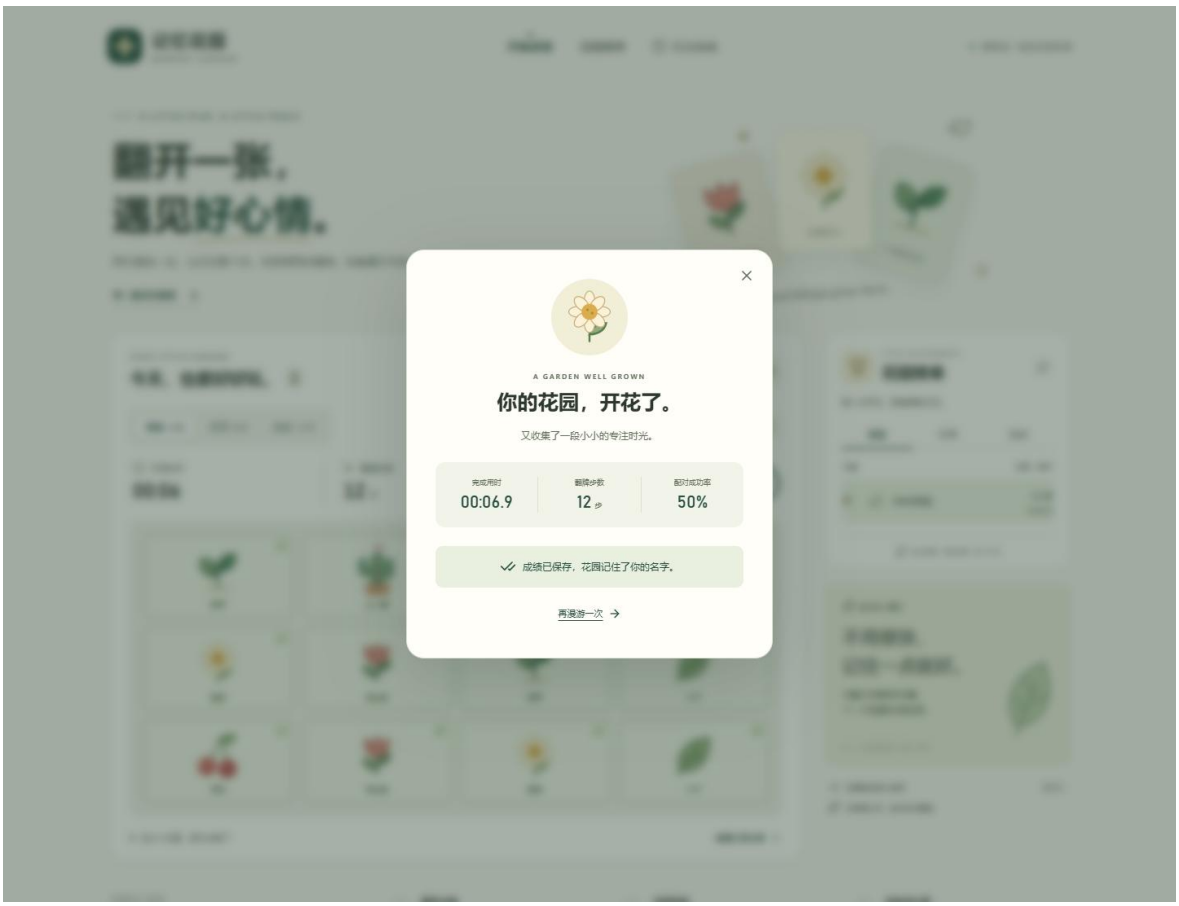


图 3　容器中完成配对并保存演示成绩

五 Docker 镜像构建与运行

5.1 多阶段构建

第一阶段使用 Node 构建 Vue，第二阶段使用 Maven 编译和打包 Spring Boot，第三阶段只复制 JAR 到 Java 运行环境。以下为实际 Dockerfile 的关键流程；完整文件位于项目根目录。

```
FROM node:24-bookworm-slim AS frontend
WORKDIR /build/frontend
COPY frontend/package*.json ./
RUN npm ci --no-audit --no-fund
COPY frontend/ ./
RUN npm run build
FROM maven:3.9.9-eclipse-temurin-21 AS backend
WORKDIR /build/backend
COPY backend/pom.xml ./
RUN mvn -B -ntp dependency:go-offline
COPY backend/src ./src
COPY --from=frontend /build/frontend/dist /build/frontend/dist
RUN mvn -B -ntp package
FROM eclipse-temurin:21-jre-jammy
# 创建普通用户与数据目录后复制 JAR
COPY --from=backend --chown=garden:garden /build/backend/target/memory-game6085.jar /app/app.jar
USER garden
ENV DATA_DIR=/app/data PORT=8080
EXPOSE 8080
ENTRYPOINT ["java", "-XX:MaxRAMPercentage=70.0", "-jar", "/app/app.jar"]
docker compose --progress plain build
```

构建日志显示 Maven BUILD SUCCESS，最终镜像命名为 memory-game6085:1.0.0。Docker CLI 显示磁盘占用约 469 MB。首次基础镜像下载曾发生短读错误，完成基础镜像拉取后重新构建成功。

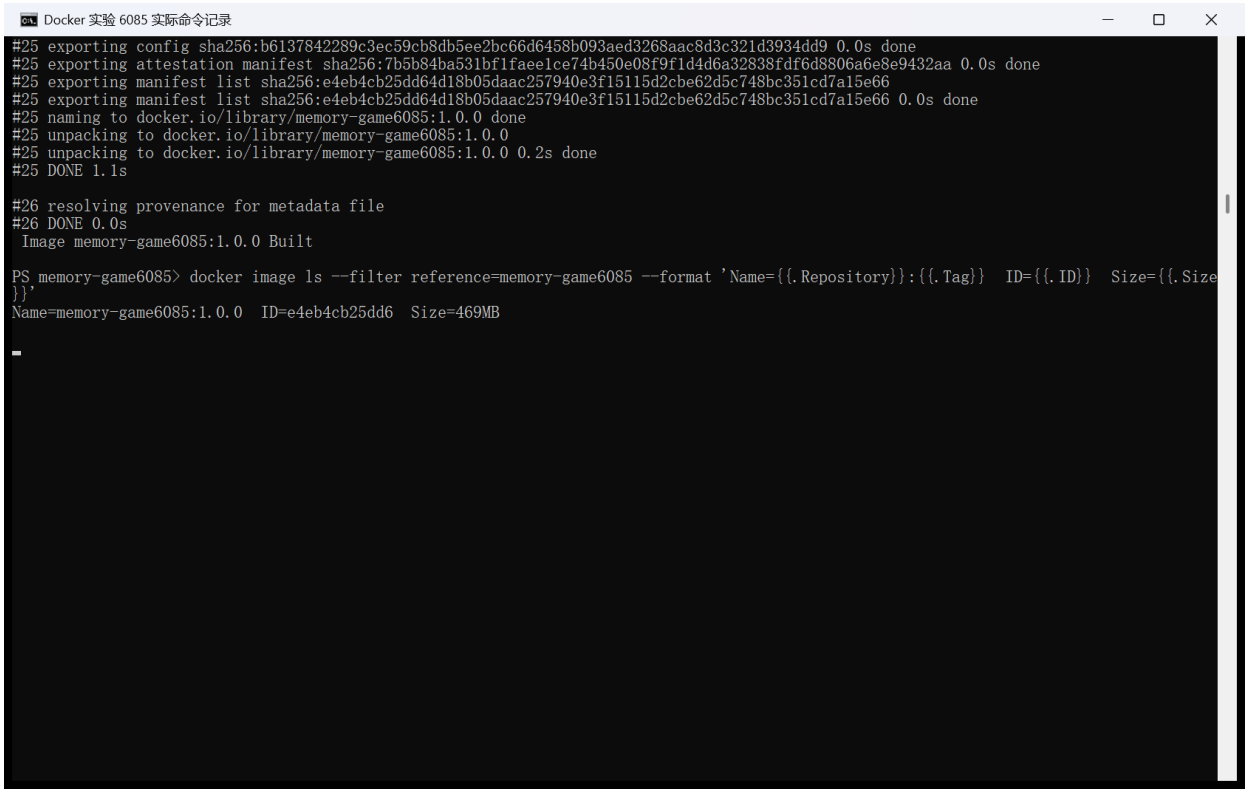


图 4　Spring Boot 打包成功并导出 Docker 镜像

5.2 启动容器与接口检查

```
docker compose up -d --no-build
docker compose ps
curl.exe -s http://localhost:18085/api/health
docker exec memory-game6085-garden-1 id
```

应用容器运行后，18085 端口可访问游戏和 API。健康检查返回 status 为 UP，database 为 SQLite。容器内运行身份为 garden，UID 和 GID 均为 999，应用不使用 root 用户运行。

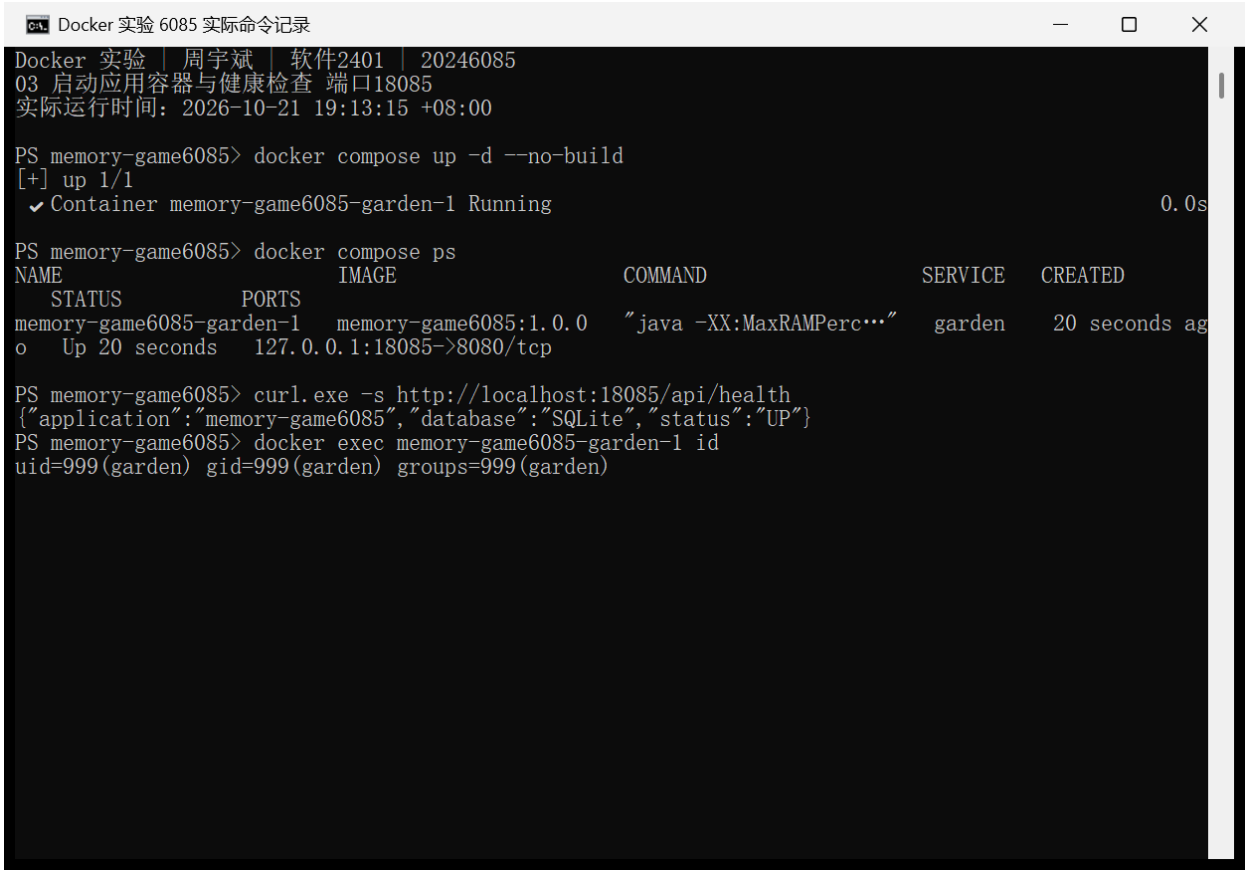


图 5　应用容器运行状态 端口映射 健康检查与普通用户

浏览器访问 http://localhost:18085 可进入游戏。该地址映射至应用容器的 8080，前端和后端同源访问，SQLite 无需额外数据库容器。

六 镜像发布与拉取

6.1 启动本机镜像仓库

本实验使用已确认的本机私有 Registry，仓库端口绑定 127.0.0.1:5000，镜像数据保存在 memory-game6085-registry-data 数据卷。仓库用于本机课程实验。

```
docker run -d --name memory-game6085-registry -p 127.0.0.1:5000:5000 -e OTEL_TRACES_EXPORTER=none --mount type=volume,source=memory-game6085-registry-data,target=/var/lib/registry registry:3
```

6.2 标记与推送镜像

```
docker tag memory-game6085:1.0.0 localhost:5000/memory-game6085:1.0.0
docker push localhost:5000/memory-game6085:1.0.0
curl.exe -s http://localhost:5000/v2/memory-game6085/tags/list
```

推送日志显示镜像层 Pushed，仓库标签查询返回 1.0.0。发布路径为 localhost:5000/memory-game6085:1.0.0。

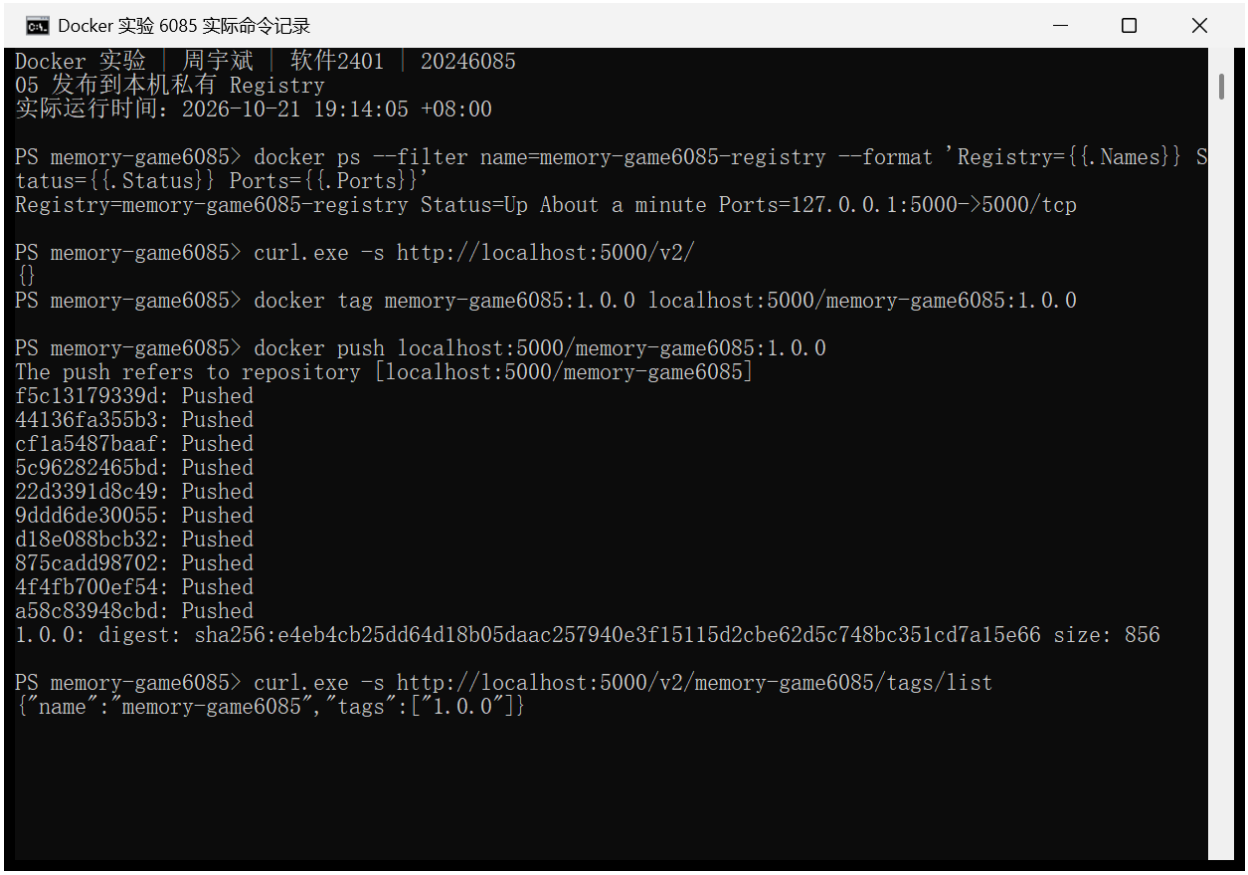


图 6　镜像实际推送成功与仓库标签查询

6.3 删除本地镜像后重新拉取

为验证镜像来自仓库，先移除本实验应用容器，保留数据卷，并删除本地的两个实验镜像标签。镜像列表确认本地不存在 memory-game6085 后，再执行 pull。

```
docker compose down
docker image rm localhost:5000/memory-game6085:1.0.0 memory-game6085:1.0.0
docker image ls
docker pull localhost:5000/memory-game6085:1.0.0
```

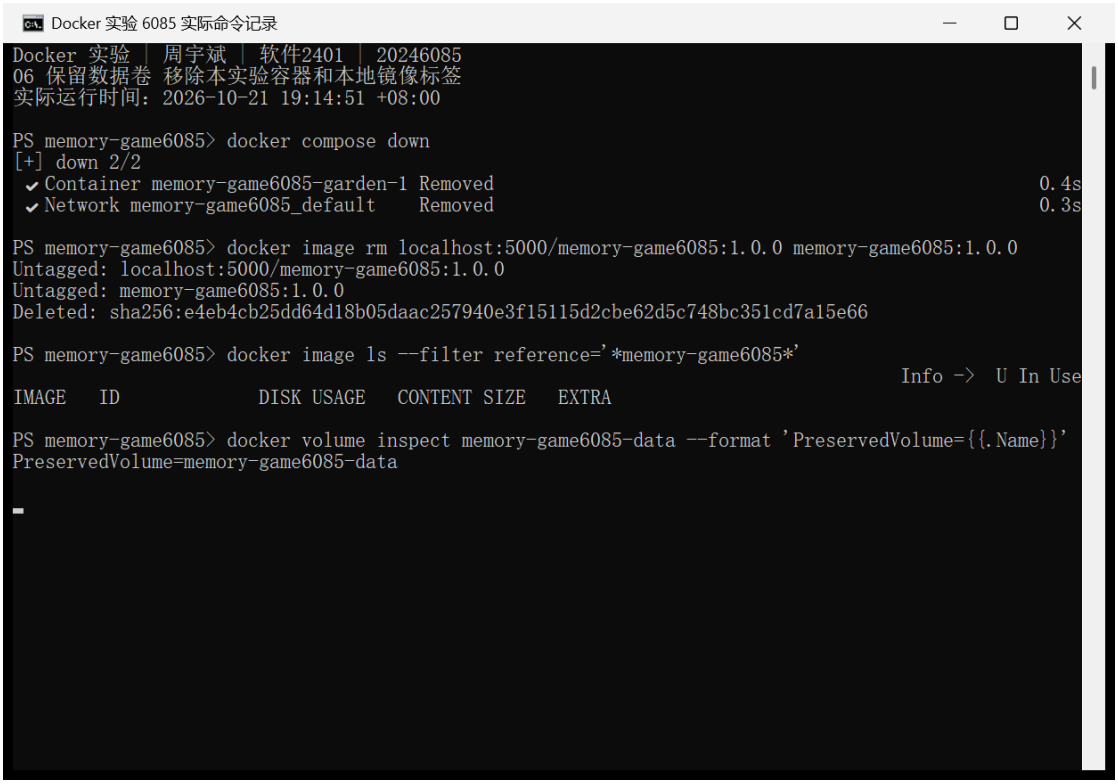


图 7　移除实验容器和本地镜像 数据卷仍然保留

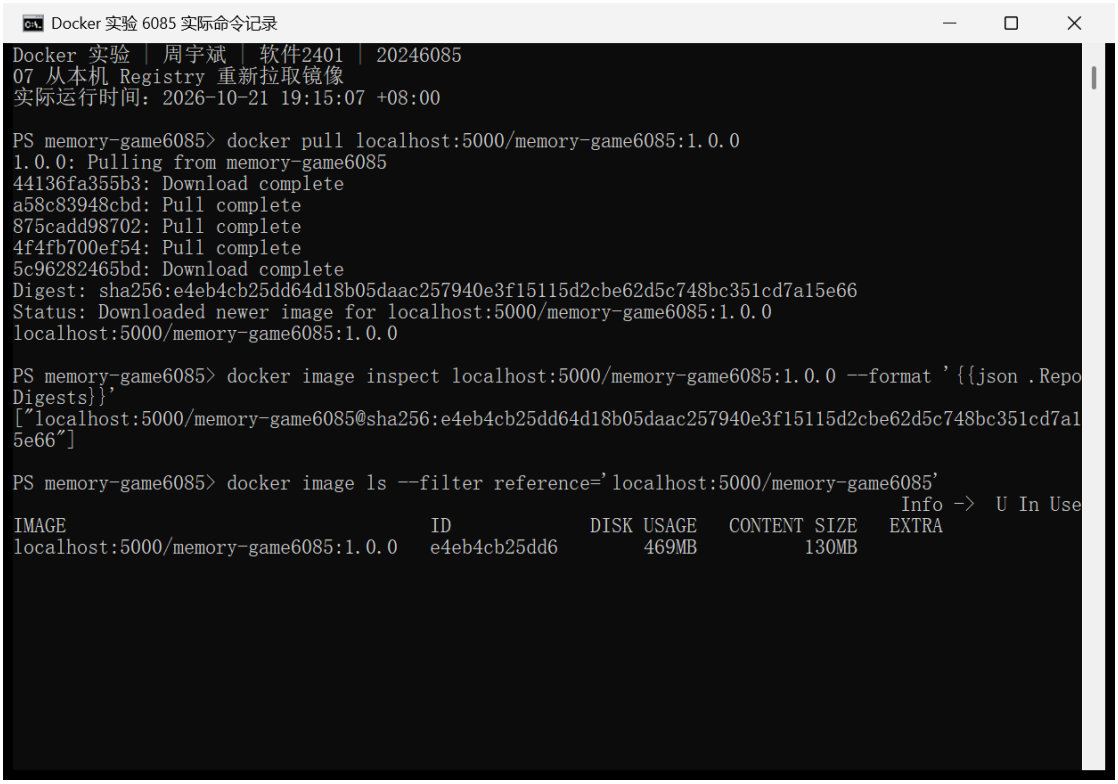


图 8　从 Registry 重新拉取成功

拉取结果为 Downloaded newer image。推送与拉取的摘要一致，说明使用的是已发布的同一版本镜像。摘要记录如下：

```
sha256:e4eb4cb25dd64d18b05daac257940e3f15115d2cbe62d5c748bc351cd7a15e66
```

七 容器重建与持久化验证

7.1 记录重建前的数据

原容器中已保存轻松难度演示成绩。实验记录了原容器标识和完整排行榜响应；移除容器时未使用删除数据卷的参数。

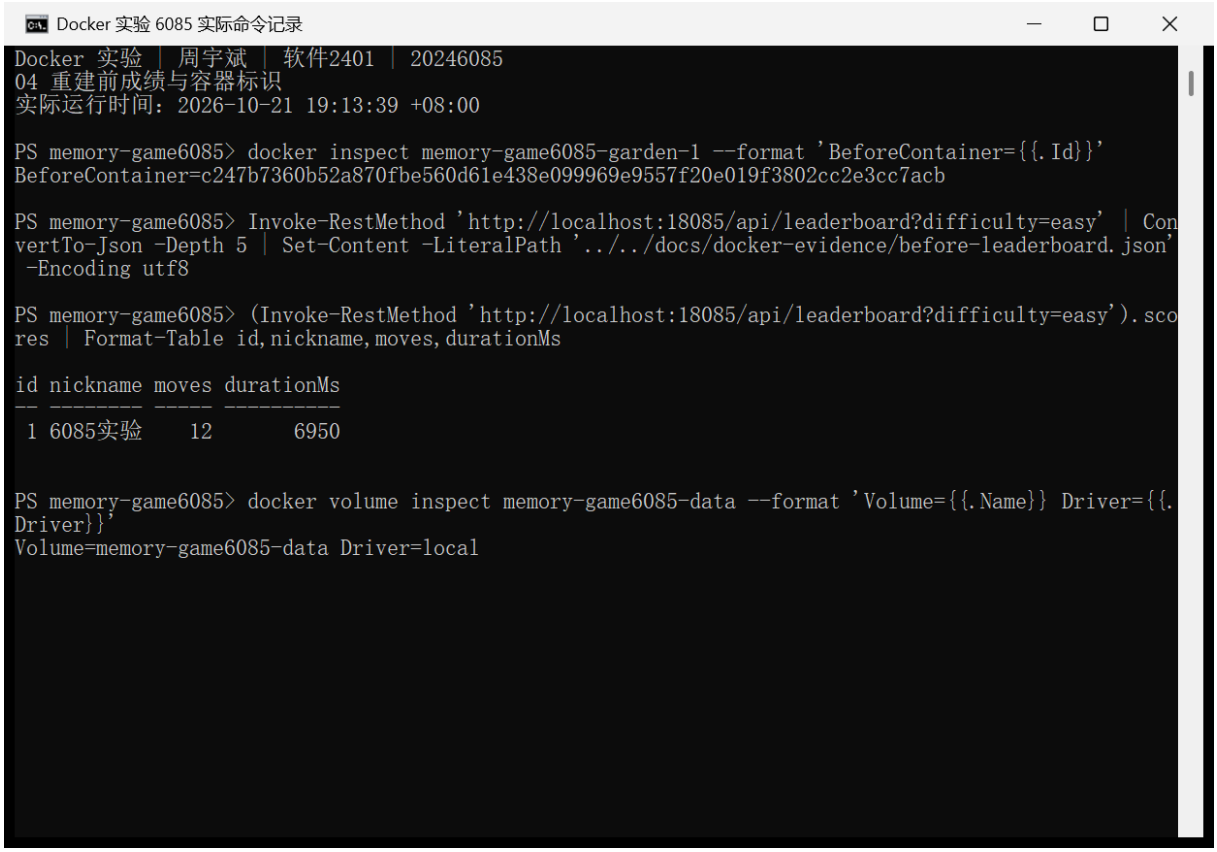


图 9　重建前的容器标识 成绩与数据卷

对比项	重建前的实际值
容器标识前 12 位	c247b7360b52
成绩编号 / 昵称	1 / 6085 实验
步数 / 用时	12 / 6950 ms
数据卷	memory-game6085-data

7.2 使用拉取镜像创建新容器

```
docker run -d --name memory-game6085-pulled -p 127.0.0.1:18085:8080 -e DATA_DIR=/app/data --mount
type=volume,source=memory-game6085-data,target=/app/data localhost:5000/memory-game6085:1.0.0
```

新容器标识前 12 位为 dd8b4bb17b50，与原容器不同。新容器直接使用拉取的仓库镜像，并继续挂载 memory-game6085-data 到 /app/data。

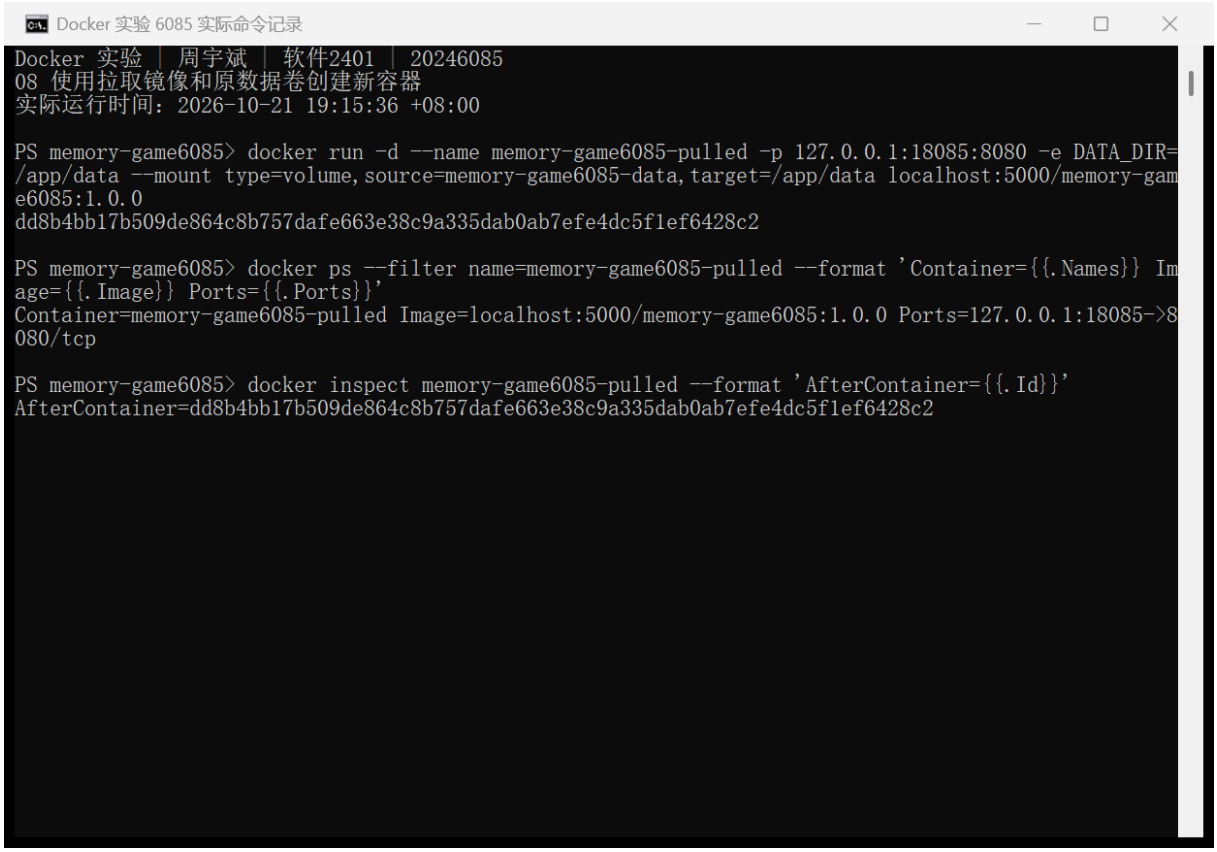


图 10　使用拉取镜像创建新容器并确认容器标识

7.3 对比重建后的成绩

新容器启动后，成绩编号、昵称、难度、步数、用时和创建时间均与重建前一致。程序对重建前后的完整排行榜 JSON 比较，结果为 LeaderboardPreserved=True。记录仍为 6085 实验、12 步、6950 ms。

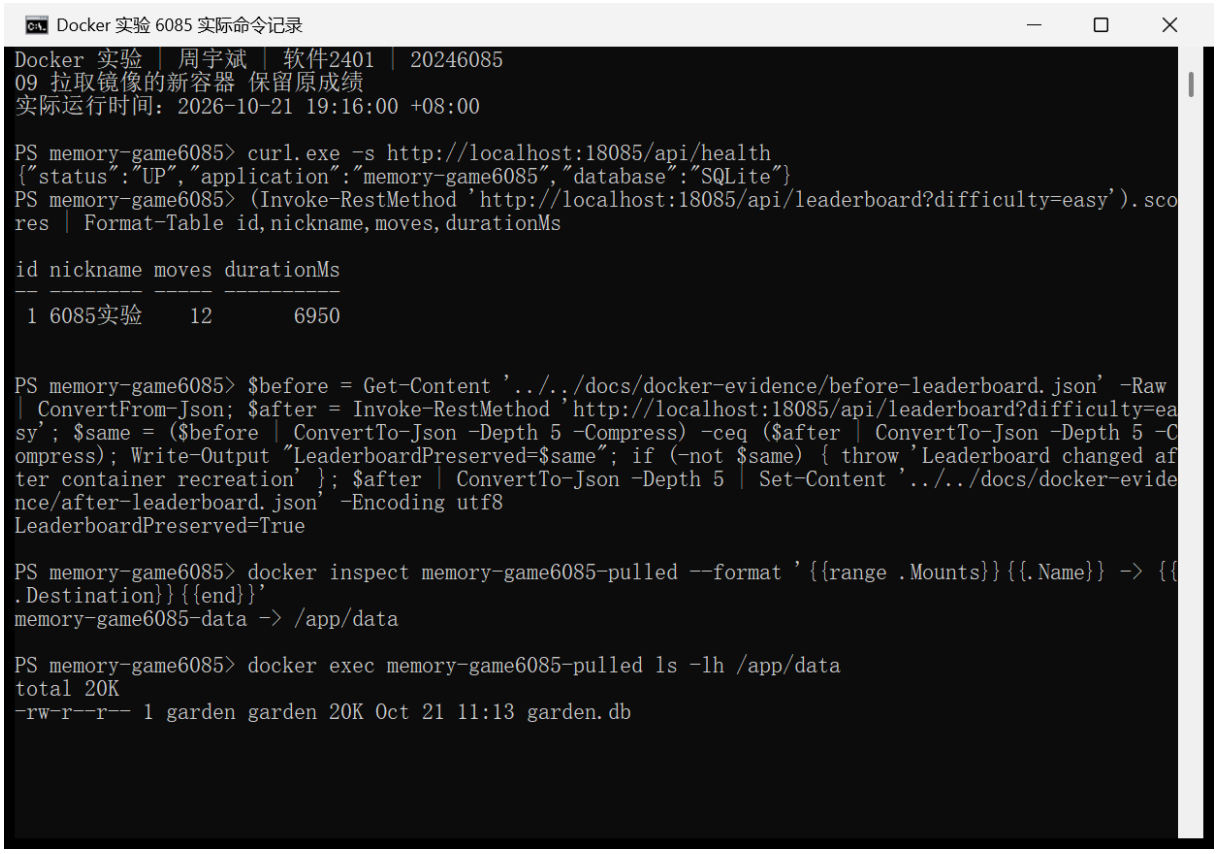


图 11　重建后榜单一致 数据卷仍挂载到同一目录

八 实验结果与问题分析

实验环节	实际结果
Docker 环境	Client / Server 正常，Linux 引擎可用
应用与镜像构建	前端构建、后端测试和 Spring Boot JAR 打包成功
容器运行	页面和 API 可访问，健康检查 UP
镜像发布	本机 Registry push 成功，可查询标签 1.0.0
镜像拉取	删除本地镜像后 pull 成功，摘要与推送一致
容器重建	新容器使用拉取镜像，容器标识与原容器不同
SQLite 数据卷	完整榜单对比一致，保存成绩保留
实验截图	真实终端窗口与容器浏览器页面均已采集

8.1 启动与端口问题

Docker Desktop 最初因遗留 socket 无法访问而退出。原运行时目录改名被 Windows 拒绝，最终通过仅为 Docker Desktop 子进程设置独立运行时缓存目录恢复启动，系统级环境变量未修改。恢复后以 Client 与 Server 响应以及实际容器结果验证环境可用。

首次构建遇到基础镜像下载中断；重新拉取所需基础镜像后完成构建。现有课堂项目占用 8080，因此将本实验宿主机端口调整为 18085，同时保留容器内部端口 8080。

8.2 实验总结

本实验完成了应用打包、镜像构建、容器访问、镜像发布与拉取，以及数据卷跨容器保留的完整验证。镜像用于分发应用程序，容器是运行实例，Registry 用于保存和分发镜像，数据卷用于保存独立于容器生命周期的数据库文件。

重建前后容器标识发生变化，但保存的成绩没有变化，说明 SQLite 文件与应用容器已经通过命名数据卷实现生命周期分离。后续复现实验时，应保持仓库地址、镜像版本、应用端口和数据卷名称一致。

九 参考资料与证据来源

原始作业要求：raw/要求.png；身份信息、技术栈和本机 Registry 方式由用户确认。

项目源代码：Code/memory-game6085/；实验结果：docs/docker-evidence/results.json；各图来自同目录的真实终端和浏览器截图。

技术资料查阅日期：2026 年 10 月 21 日。

Vue 官方文档：https://vuejs.org/guide/essentials/reactivity-fundamentals.html

Vite 官方文档：https://vite.dev/guide/

Spring Boot 官方文档：https://docs.spring.io/spring-boot/maven-plugin/packaging.html

SQLite JDBC 官方仓库：https://github.com/xerial/sqlite-jdbc

Docker 数据卷官方文档：https://docs.docker.com/engine/storage/volumes/

CNCF Distribution 官方部署文档：https://distribution.github.io/distribution/about/deploying/